

Quantifier Elimination Meets Treewidth

Hao Wu^{*1,2} , Jiyu Zhu^{*1,2} , Amir Kafshdar Goharshady³ ,
Jie An^{4,2} , Bican Xia⁵ , and Naijun Zhan^{6,7}  

¹ KLSS, Institute of Software Chinese Academy of Sciences, China

² University of Chinese Academy of Sciences, China

³ University of Oxford, United Kingdom

⁴ National Key Laboratory of Space Integrated Information System,
Institute of Software Chinese Academy of Sciences, China

⁵ School of Mathematical Sciences, Peking University, China

⁶ School of Computer Science, Peking University, China

⁷ Zhongguancun Laboratory, China

{wuhao, zhujy}@ios.ac.cn amir.goharshady@cs.ox.ac.uk
anjie@iscas.ac.cn xbc@math.pku.edu.cn njzhan@pku.edu.cn

Abstract. In this paper, we address the complexity barrier inherent in Fourier-Motzkin elimination (FME) and cylindrical algebraic decomposition (CAD) when eliminating a block of (existential) quantifiers. To mitigate this, we propose exploiting structural sparsity in the variable dependency graph of quantified formulas. Utilizing tools from parameterized algorithms, we investigate the role of *treewidth*, a parameter that measures the graph’s tree-likeness, in the process of quantifier elimination. A novel dynamic programming framework, structured over a tree decomposition of the dependency graph, is developed for applying FME and CAD, and is also extensible to general quantifier elimination procedures. Crucially, we prove that when the treewidth is a constant, the framework achieves a significant exponential complexity improvement for both FME and CAD, reducing the worst-case complexity bound from doubly exponential to single exponential. Preliminary experiments on sparse linear real arithmetic (LRA) and nonlinear real arithmetic (NRA) benchmarks confirm that our algorithm outperforms the existing popular heuristic-based approaches on instances exhibiting low treewidth.

Keywords: Quantifier Elimination · Fourier-Motzkin Elimination · Cylindrical Algebraic Decomposition · Parameterized Algorithms · Treewidth

1 Introduction

Quantifier elimination (QE) is a fundamental technique in mathematical logic that transforms a first-order formula containing existential ($\exists$) and universal ($\forall$) quantifiers into an equivalent quantifier-free formula. A theory that admits a

The first two authors marked with $\star$ contributed equally to this work and should be considered co-first authors.

quantifier elimination procedure is highly desirable, as its decidability immediately reduces to the decidability of its quantifier-free fragment. This theoretical significance strongly motivates researchers in mathematics and computer science to investigate various theories that admit quantifier elimination procedures. This paper primarily focuses on quantifier elimination within two central theories over the real numbers: *linear real arithmetic* (LRA), where atomic formulas are defined by strictly linear constraints, and *nonlinear real arithmetic* (NRA), which permits general polynomial constraints.

For LRA, the *Fourier-Motzkin elimination* (FME) algorithm is the first quantifier elimination procedure, originally proposed by Fourier [33] and rediscovered by Motzkin [60]. This algorithm can be understood as reducing a system of linear inequalities by removing variables one by one. Its importance primarily stems from its geometric interpretation, which corresponds to the projection of a polyhedron (described by linear inequalities) onto lower-dimensional subspaces [67, Sect. 12.2]. However, a significant challenge of applying the FME algorithm is its high worst-case complexity, which is doubly exponential in the number of quantified variables. This high complexity results from the rapid, often redundant, proliferation of new constraints during the elimination steps [43,44,47]. For eliminating a block of existential quantifiers, recent works [61,62] provide a divide-and-conquer approach to improve the complexity to single exponential.

For NRA, the first quantifier elimination procedure was developed around the 1930s by Tarski in his seminal work [72]. While Tarski’s procedure is not elementary recursive, the first elementary recursive quantifier elimination procedure was developed by Collins in 1975 [20], known as *cylindrical algebraic decomposition* (CAD). The CAD algorithm marks a milestone in real algebraic geometry and a comprehensive survey can be found in [16]. Ever since its origin, the CAD algorithm, as well as its various variants [59,39,21,13,18,40,15,38,68,10,37], has remained the most authoritative procedure for NRA. It has found broad application in critical domains, including formal verification [50,65,54,34,1], control synthesis [28,49,41], and hybrid system analysis [2,69]. Similar to the FME algorithm, the CAD algorithm also suffers from a worst-case doubly exponential complexity, but in the number of *all* occurring variables [23]. Besides the complexity of internal algebraic operations, it is well known that the order of variables to be eliminated, called *variable elimination ordering*, has a huge impact on the practical performance when eliminating a block of existential quantifiers. Therefore, many heuristics based on sophisticated structural analysis [14,26,11,53] and machine learning techniques have been applied to this task [42,31,19,46].

While existing works mostly focus on improving FME and CAD for general inputs, a natural but unexplored (up to our knowledge) problem is: **Can we design strategies for certain classes of inputs with better worst-case complexity upper bounds?** Specifically, in this paper, we consider posing restrictions on the structural complexity of the dependency relationship of variables. This relationship is typically represented by a graph, precisely the *primal graph*, whose nodes correspond to variables of the input formula, and two vari-

ables are linked if and only if they both occur in an atomic formula. To study this problem, we utilize tools coming from *parameterized algorithms*.

The study of parameterized algorithms offers a fine-grained analytical framework for tackling computationally hard problems by introducing a secondary measurement, k , known as the parameter [29,22]. For example, a problem is classified as *fixed-parameter tractable* (FPT) if it admits an algorithm with a time complexity of $O(f(k) \cdot n^c)$, where n is the input size, f is a computable function depending solely on k , and c is a constant independent of n . In general, the goal of parameterized complexity is to conduct a finer-grained analysis of complexity by isolating the combinatorial explosion to a specific parameter.

When the input is a graph, *treewidth* is one of the most important parameters to measure its inherent complexity. Intuitively, treewidth quantifies how “tree-like” a graph is: a smaller treewidth value indicates a greater sparsity in the graph structure. The main advantage of treewidth is that a bounded treewidth allows a vast collection of classical NP-hard problems to become fixed-parameter tractable [64,8,22]. This tractability is fundamentally realized by employing *dynamic programming* (DP) techniques executed over a tree structure related to the graph, called a *tree decomposition* [6]. For constraint satisfiability problems, treewidth measures the structural complexity of variable dependencies and has been extensively studied [70,57,32,27,35,55]. However, the role of treewidth in the quantifier elimination of first-order logic theories is largely untouched, possibly due to the inherent high complexity of these procedures.

Contributions. In this paper, we develop a novel framework to exploit the treewidth sparsity pattern in the process of applying quantifier elimination procedures to eliminate a block of (existential) quantifiers. Here, the treewidth sparsity pattern means that the primal graph of the input formula has a small treewidth. The primary contributions include:

- We propose a dynamic programming algorithm for applying FME and CAD to eliminate a block of quantifiers. It is executed over a tree decomposition of the formula’s primal graph, where the treewidth of the graph determines the structure of this decomposition. We prove its correctness and demonstrate its extensibility to general quantifier elimination procedures.
- We prove that, when the treewidth is a constant, the worst-time complexity of FME and CAD can be *exponentially improved* from doubly exponential to single exponential in the number of *eliminated* variables and *all occurring* variables, respectively. The core idea is the utilization of a special tree decomposition structure, called *balanced tree decompositions* [17].
- We conduct experiments on randomly generated LRA and NRA benchmarks exhibiting sparse patterns. Experimental results indicate that our algorithm outperforms other popular heuristics on problem instances with low treewidth.

Our starting point is a parameterized algorithmic perspective, a methodology that is orthogonal to most existing techniques. The most closely related work is [53], which exploits the chordal structure of the primal graph to guide the

variable elimination ordering in CAD. Since chordal graphs and tree decompositions are two sides of one coin [25, Sec. 12.3], our algorithm in Sect. 3 can be viewed as a dual version of theirs. Nevertheless, utilizing treewidth offers significant algorithmic advantages. Specifically, it enables a dynamic programming framework that is more comprehensible and, by drawing upon concepts from parameterized complexity theory, a more fine-grained analysis of the complexity issues involved.

Outline. The rest of this paper is organized as follows: Sect. 2 introduces necessary concepts. Sect. 3 presents the dynamic programming algorithm for FME and presents how to extend it to CAD and general quantifier elimination procedures. Sect. 4 analyzes the complexity of our framework for FME and CAD under the assumption that the treewidth is a constant. Sect. 5 reports the experimental results and Sect. 6 finally concludes the paper.

2 Preliminaries

Let $\mathbb{R}$ and $\mathbb{N}$ denote the set of real numbers and natural numbers, respectively. A vector of n real variables is denoted by $(x_1, \dots, x_n) \in \mathbb{R}^n$, also written as $\mathbf{x}$ for short. We denote by $\langle x_{\sigma(1)}, \dots, x_{\sigma(n)} \rangle$ a linear ordering among these variables, where σ is from the permutation group of size n . For example, $\langle x_1, \dots, x_n \rangle$ denotes the natural order from x_1 to x_n . We assume readers are familiar with first-order logic and recommend the book [12] for reference. For a set C of atomic formulas, we denote by $\bigwedge C$ the conjunction of all atomic formulas in C .

2.1 LRA, NRA, and Quantifier Elimination

Quantifier Elimination (QE). We say a first-order theory $\mathcal{T}$ admits *quantifier elimination* if there exists an algorithm, called a quantifier elimination procedure, that transforms a given quantified $\mathcal{T}$ -formula into an equivalent formula without quantifiers. Formally, w.l.o.g., we consider the input formula Φ to be a conjunction of atomic formulas that is existentially quantified:

$$\Phi \triangleq \exists x_m, \dots, \exists x_1. \bigwedge_i \varphi_i(x_1, \dots, x_n), \quad (1)$$

where $m, n \in \mathbb{N}$, $m \leq n$, and each φ_i is an atomic $\mathcal{T}$ -formula. The output of the quantifier elimination procedure is a quantifier-free formula in variables $x_{m+1}, \dots, x_n$ that is equivalent to Φ . For formulas with quantifier alternations (e.g., $\forall x_2 \exists x_1$), we eliminate quantifiers from inside out, eliminating a block of quantifiers of the same type at each step. In what follows, we primarily focus on the FME algorithm for LRA and on the CAD algorithm for NRA.

Linear Real Arithmetic (LRA). LRA refers to the first-order theory with signature $\{0, 1, +, <\}$ and domain $\mathbb{R}$. Given variables $(x_1, \dots, x_n) \in \mathbb{R}^n$ and real constants $(a_1, \dots, a_n, b) \in \mathbb{R}^{n+1}$, an LRA atomic formula is of the form

$a_1x_1 + \dots + a_nx_n - b \bowtie 0$, where the relation symbol $\bowtie \in \{<, >, =, \leq, \geq\}$. The semantics of LRA is interpreted in the standard way.

Fourier-Motzkin Elimination (FME). The FME algorithm takes a pair (C, x_r) as input, where C is a set of LRA atomic formulas and x_r is a variable occurring in these constraints, and proceeds one of the following two steps:

- (I) If x_r appears in an equality constraint of the form $\sum_{j=1}^n a_{i,j} \cdot x_j - b_i = 0$ with $a_{i,r} \neq 0$, the procedure removes this constraint from C and outputs C by replacing every occurrence of x_r with $\frac{b_i}{a_{i,r}} - \sum_{j=1}^{r-1} \frac{a_{i,j}}{a_{i,r}} \cdot x_j - \sum_{j=r+1}^n \frac{a_{i,j}}{a_{i,r}} \cdot x_j$.
- (II) If x_r only appears in inequality constraints of the form $\sum_{j=1}^n a_{i,j} \cdot x_j - b_i \bowtie 0$, where $\bowtie$ is not $=$ and $a_{i,r} \neq 0$, for each of such constraints we derive a term $\frac{b_i}{a_{i,r}} - \sum_{j=1}^{r-1} \frac{a_{i,j}}{a_{i,r}} \cdot x_j - \sum_{j=r+1}^n \frac{a_{i,j}}{a_{i,r}} \cdot x_j$ that is called a *bound* of x_r . Moreover, the bound is called an upper bound if $a_{i,r} > 0$ and a lower bound if $a_{i,r} < 0$. In either case, it is called strict if $\bowtie \in \{<, >\}$. Let U_r and L_r denote the set of upper bounds and lower bounds of x_r , respectively. Then, the procedure outputs the set of constraints $\{l \triangleleft u \mid l \in L_r, u \in U_r\}$, where $\triangleleft$ is $<$ if both u and l are strict and is $\leq$ otherwise.

We denote the output by $\text{FME}(C, x_r)$, which is a set of atomic formulas without x_r . If C is taken to be the set of atomic formulas of an LRA formula Φ of the form Eq. (1), then Φ is equivalent to the following quantifier-free formula $\bigwedge \text{FME}(C, \langle x_1, \dots, x_m \rangle)$, where $\text{FME}(C, \langle x_1, \dots, x_m \rangle)$ denotes the output of recursively applying FME procedure on C w.r.t. the ordering from x_1 to x_m . For a more detailed description of the algorithm, please refer to [52, Sec. 5.4].

Nonlinear Real Arithmetic (NRA). NRA refers to the first-order theory with signature $\{0, 1, +, \cdot, <\}$ and domain $\mathbb{R}$. A monomial is a product of powers of variables with nonnegative integer exponents, denoted by $\mathbf{x}^\alpha = x_1^{\alpha_1} \dots x_n^{\alpha_n}$ for some $\alpha \in \mathbb{N}^n$. A polynomial is a finite summation of monomials $\sum_{\alpha} c_{\alpha} \mathbf{x}^\alpha$, where $c_{\alpha} \in \mathbb{R}$ are called coefficients. The set of polynomials in variables $\mathbf{x}$ with real coefficients is denoted by $\mathbb{R}[\mathbf{x}]$. An NRA atomic formula is of the form $f(\mathbf{x}) \bowtie 0$, where $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$ and $\bowtie \in \{<, >, =, \leq, \geq\}$. It is clear that an LRA constraint is a special case of an NRA constraint where the polynomial f is linear, i.e., of degree 1. The semantics of NRA are also interpreted in the standard way.

Cylindrical Algebraic Decomposition (CAD). Given an NRA formula Φ of the form Eq. (1) and letting $P \subset \mathbb{R}[x_1, \dots, x_n]$ denote the set of polynomials occurring in Φ , the key idea underlying CAD is to decompose the state space $\mathbb{R}^n$ into a finite number of regions called *cells*. This decomposition satisfies two primary properties: (i) in each cell each polynomial in P remains a constant sign, and (ii) the boundary of each cell can be derived from polynomials in P . Due to the first property, the sign of a polynomial in P over a cell can be determined by a single sample point within that cell. Consequently, a finite set of sample points from the decomposition is sufficient to determine in which cells the conjunction of atomic formulas in Φ evaluates to true. Finally, by utilizing the formulas that define these cells, we can construct a quantifier-free formula equivalent to Φ .

The CAD algorithm consists of two phases, *projection* and *lifting*, while in this paper, *we solely focus on the projection phase*. The projection phase takes a variable ordering, say $\langle x_1, x_2, \dots, x_n \rangle$, where quantified variables precede free variables, and constructs a sequence of polynomial sets

$$\{P_1(=P), P_2, \dots, P_n\} \text{ with } P_i \subset \mathbb{R}[x_i, \dots, x_n] \quad (2)$$

through the iterative application of a projection operator Proj . Each subsequent set $P_{i+1} = \text{Proj}(P_i, x_i)$ is defined in a lower-dimensional space by removing the variable x_i and carries enough information about P_i . There exist several different definitions of the projection operator, and in our analysis, we use McCallum’s projection operator [59]. After the projection phase, the lifting phase then constructs the sign-invariant cells from 1-dimension to higher dimensions. We provide a more detailed description of the CAD algorithm [4, 48, 66] in Appx. A.1, which may not affect the understanding of this paper.

2.2 Treewidth and Tree Decomposition

In this part, we introduce basic concepts related to the sparsity of graphs. We recommend referring to [22, Ch. 7] for an in-depth investigation of this topic.

Graph and Primal Graph. We use $G = (V, E)$ to denote an undirected graph, where V is a finite set of vertices and $E \subset V \times V$ is the set of edges. The size of a graph, denoted $|G|$, is the cardinality of V . A clique in a graph is a subset of mutually adjacent vertices. Given a formula Φ in the form of Eq. (1), the *primal graph* $G_\Phi = (V_\Phi, E_\Phi)$ is a graph associated to Φ such that $V_\Phi = \{x_1, \dots, x_m\}$ is the collection of (*existentially*) *quantified variables* and $(x_{j_1}, x_{j_2}) \in E_\Phi$ if and only if both $a_{i,j_1} \neq 0$ and $a_{i,j_2} \neq 0$ for some index i of atomic formulas, i.e., x_{j_1}, x_{j_2} occur simultaneously in an atomic formula. Throughout this paper, we will always assume the primal graphs are connected; otherwise, each connected component can be processed separately.

Example 1 (Running Example). Consider an LRA formula Φ of the form Eq. (1) and contains existentially quantified variables $(x_1, \dots, x_8)$. The portion of Φ related to x_1 is displayed in Fig. 1; the full formula containing 20 atomic formulas is in Appx. A.2. The atomic formulas in Φ display a certain “sparsity” pattern—each variable only occurs in a small portion of atomic formulas. For example, variable x_1 only occurs in three atomic formulas, namely $x_1 + 2x_2 + 3x_3 \leq 20$, $x_1 - x_2 + 2x_3 \geq -5$, and $x_1 - 4x_2 \leq 0$. The primal graph of Φ is shown in Fig. 2.

Tree. A tree $T = (V, E)$ is an undirected graph that contains no cycles, i.e., there is only one unique path between any two vertices. For clarity, the vertices of a tree will be called *nodes*. We always assume that a tree under consideration is *rooted*, i.e., a node $r \in V$ is designated as the root. The *level* of a node $v \in V$, denoted $\text{Lv}(v)$, is the length of the simple path from r to v . The *height* of a tree is the maximum level among all nodes. The *parent* of a node v is the node connected to v on the path to the root, and a child of a node v is a node of which

$$\Phi \triangleq \exists x_1, \dots, x_8. \left(\begin{array}{l} x_1 + 2x_2 + 3x_3 \leq 20 \\ \wedge x_1 - x_2 + 2x_3 \geq -5 \\ \wedge x_1 - 4x_2 \leq 0 \\ \wedge \dots (\text{constraints without } x_1) \end{array} \right)$$

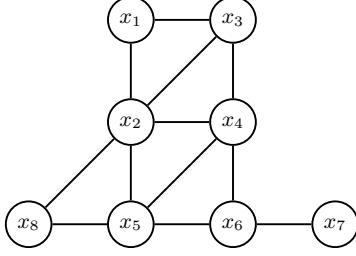
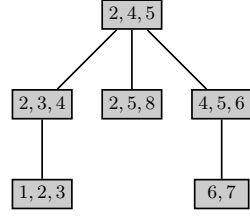
Fig. 1: An LRA formula Φ Fig. 2: The primal graph of Φ 

Fig. 3: A tree decomposition of Fig. 2

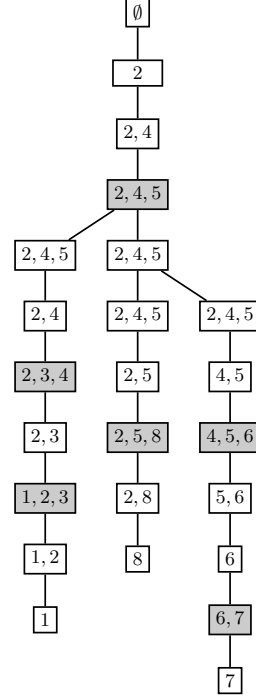


Fig. 4: The nice tree decomposition of Fig. 3. Gray nodes correspond to the nodes in Fig. 3

v is the parent. A *leaf* is a node with no children. A tree is called *binary* if each node has at most two children. For each node v in T , we denote by T_v the subtree rooted at v ; the nodes in T_v (excluding v itself) are called the *descendants* of v .

Tree Decomposition and Treewidth [36,63,64]. Given an undirected graph $G = (V, E)$, a *tree decomposition* of G is a tree $T = (\mathcal{B}, E_T)$ satisfying the following conditions: (1) Each node $b \in \mathcal{B}$ of T , also called a *bag*, is associated to a subset of the vertex set $V_b \subseteq V$; (2) The bags cover the entire vertex set V of G , i.e., $\bigcup_{b \in \mathcal{B}} V_b = V$; (3) For every edge $e = (u, v) \in E$, there exists a bag b such that both $u \in V_b$ and $v \in V_b$; and (4) For every three bags $b_1, b_2, b_3 \in \mathcal{B}$, if b_3 is on the path from b_1 to b_2 in T , then $V_{b_1} \cap V_{b_2} \subseteq V_{b_3}$ (equivalently, every vertex $v \in V$ appears in a connected subtree of T).

The *width* of a tree decomposition T , denoted by $w(T)$, is defined to be the size of the largest bag minus one, i.e., $w(T) = \max_{b \in \mathcal{B}} |V_b| - 1$. The *treewidth* of the graph G , denoted by $tw(G)$, is defined to be the minimum width among all possible tree decompositions of G . A tree decomposition with width $tw(G)$ is called *optimal*. For example, Fig. 3 presents an optimal tree decomposition of the primal graph in Exmp. 1 of width 2. The numbers in the labels refer to

the variable indexes, for instance, the node labeled $\{1, 2, 3\}$ in Fig. 3 represents the bag $\{x_1, x_2, x_3\}$. For a primal graph G_Φ , since every clique is contained in at least one bag in its tree decomposition [22], the largest number of quantified variables in an atomic formula gives a lower bound for $tw(G_\Phi)$.

A smaller treewidth indicates a greater sparsity. In general, it is NP-complete to determine whether a given graph G has treewidth at most a given variable k [3]. However, Bodlaender showed that when k is fixed, there exists a linear-time algorithm to test whether G has treewidth at most k , and if so, to construct an optimal tree-decomposition with $O(n)$ nodes [7]. In this paper, we rely on an external algorithm [71] to compute tree decompositions. There are also many other well-optimized tools, see [24] for a survey.

Nice Tree Decomposition [51]. A tree decomposition $T = (\mathcal{B}, E_T)$ of a graph G is *nice* if T is rooted at a bag $r \in \mathcal{B}$ with $V_r = \emptyset$ and each bag of T is of one of the following four types: (1) *leaf* bag is a bag b having no children and $|V_b| = 1$; (2) A *join* bag is a bag b having exactly two children b_1 and b_2 such that $V_b = V_{b_1} = V_{b_2}$; (3) An *introduce* bag is a bag b having exactly one child b' such that $|V_b| = |V_{b'}| + 1$ and $V_{b'} \subseteq V_b$; (4) A *forget* bag is a bag b having exactly one child b' such that $|V_b| = |V_{b'}| - 1$ and $V_b \subseteq V_{b'}$. As an example, Fig. 4 gives a nice tree decomposition based on the tree decomposition in Fig. 3.

The concept of nice tree decomposition is introduced as a canonical form for representing tree decompositions. When the treewidth is a constant, a tree decomposition can be transformed into a nice tree decomposition in linear time, with the same width but a linear increase in size [22, Lem. 7.4], which helps design dynamic programming algorithms and conduct computational analysis.

3 Exploiting Treewidth Sparsity in Quantifier Elimination

Now we show how to exploit the treewidth sparsity of the primal graph in quantifier elimination. In Sect. 3.1, we explain our main idea using Exmp. 1 and present our dynamic programming framework for FME based on tree decomposition. Then we extend the framework to CAD and the general quantifier elimination procedures in Sect. 3.2. The formal analysis of the complexity improvement brought by the treewidth sparsity will be presented in Sect. 4.

3.1 A Dynamic Programming Framework for FME

To exploit the sparsity pattern within the primal graph, we begin by showing that the FME algorithm can be understood as a vertex elimination operation. Let us consider applying the FME algorithm to our running example Exmp. 1 to eliminate the variable x_1 . We first rewrite the three atomic formulas containing x_1 into the following forms, separating x_1 from other terms: $x_1 \leq 20 - 2x_2 - 3x_3 \wedge -5 + x_2 - 2x_3 \leq x_1 \wedge x_1 \leq 4x_2$, which give upper bounds and lower bounds for x_1 . Then, the FME algorithm removes x_1 by requiring that each lower bound is less than each upper bound. This results in two atomic formulas $-5 + x_2 - 2x_3 \leq 20 - 2x_2 - 3x_3 \wedge -5 + x_2 - 2x_3 \leq 4x_2$, which only involve x_2 and

x_3 . Here, a straightforward yet important observation is that the FME algorithm is performed *locally*—when a variable is eliminated (e.g., x_1), the variables in the newly introduced atomic formulas will be exactly its neighbors in the primal graph (e.g., x_2 and x_3). In other words, eliminating variables from a formula corresponds to removing vertices from the primal graph, accompanied by the introduction of new atomic formulas about the neighbor.

Using tree decompositions, we can re-explain the process of eliminating x_1 in a tree-like manner. Consider the nice tree decomposition presented in Fig. 4, the process of eliminating x_1 consists of two steps. In the first step, we assign each atomic formula of Φ to a bag in the nice tree decomposition depending on the involved variables. This is described by the initialization of a function $\mathcal{I}$ that maps each bag to a set of atomic formulas. For example, for the three bags located at the left-most branch of the nice tree decomposition in Fig. 4, we set $\mathcal{I}(\{x_1\}) \leftarrow \{\}$ (meaning there is no atomic formulas containing only x_1), $\mathcal{I}(\{x_1, x_2\}) \leftarrow \{x_1 - 4x_2 \leq 0\}$, and $\mathcal{I}(\{x_1, x_2, x_3\}) \leftarrow \{x_1 - x_2 + 2x_3 \geq -5, x_1 + 2x_2 + 3x_3 \leq 20\}$. Note that the atomic formula $x_1 - 4x_2 \leq 0$ can also be assigned to $\mathcal{I}(\{x_1, x_2, x_3\})$ without affecting the result.

In the second step, we propagate the atomic formulas stored in $\mathcal{I}$ from bottom up, by recursively computing another function $\mathcal{V}$. For example, for the leaf bag $\{x_1\}$, we set $\mathcal{V}(\{x_1\}) = \mathcal{I}(\{x_1\}) = \{\}$. For the introduce bag $\{x_1, x_2\}$, we set $\mathcal{V}(\{x_1, x_2\}) \leftarrow \mathcal{I}(\{x_1, x_2\}) \cup \mathcal{V}(\{x_1\}) = \{x_1 - 4x_2 \leq 0\}$. The computation of $\mathcal{V}(\{x_1, x_2, x_3\})$ is similar. When the forget bag $\{x_2, x_3\}$ is reached, we set $\mathcal{V}(\{x_2, x_3\}) \leftarrow \mathcal{I}(\{x_2, x_3\}) \cup \text{FME}(\bigwedge \mathcal{V}(\{x_1, x_2, x_3\}), x_1)$, which corresponds an application of the FME algorithm w.r.t. the variable removed at this bag.

By repeating the above operations, the execution of the FME algorithm can be described through a dynamic programming framework, as shown in Alg. 1, which consists of three steps.

Step 1: Constructing a nice tree decomposition (Line 1–3). To exploit the sparsity of the input formula, the algorithm first computes a tree decomposition of the formula’s primal graph, by invoking some external tree decomposition algorithms. Different choices of the algorithms lead to different tree decompositions and hence impact the overall computational complexity, which will be detailed later in Sect. 4 and Sect. 5. Nevertheless, our framework works as long as $T = (\mathcal{B}, E_T)$ in Line 2 is a nice tree decomposition.

Step 2: Initialization (Line 4–17). Based on the nice tree decomposition T , we can perform the FME algorithm in a dynamic programming style. To achieve this, we define two value functions, $\mathcal{I}$ and $\mathcal{V}$, that map bags in T into sets of atomic formulas. In this step, the value function $\mathcal{I}$ is initialized from top down: when a bag $b \in T$ is visited, all atomic formulas φ with $\text{var}(\varphi) \subseteq b$ are collected in $\mathcal{I}(b)$ and removed from C . According to the definition of tree decompositions, it is straightforward to see that the following properties hold: Each atomic formula $\varphi \in C$ is assigned to a bag b such that $\text{var}(\varphi) \subseteq b$ and $\text{Lv}(b)$ is minimized. We note that, when adding the children of a bag b into the queue in Line 10, we can break ties with any user-specified heuristics. The selection

Algorithm 1: A Dynamic Programming Framework for FME

Input : a LRA formula Φ of the form Eq. (1);
Output: a quantifier-free formula equivalent to Φ

```

1  $\triangleright$  Invoke external algorithms to compute nice tree decompositions.
2 Construct the primal graph  $G_\Phi = (V_\Phi, E_\Phi)$  of  $\Phi$ ;
3 Compute a nice tree decomposition  $T = (\mathcal{B}, E_T)$  of  $G_\Phi$ ;
4  $\triangleright$  Initialize  $\mathcal{I}$  from top down.
5  $C \leftarrow$  atomic formulas in  $\Phi$ ;
6 Initialize an empty queue of bags  $q = \langle \rangle$ ;
7  $q.\text{enqueue}(r_T)$ ; // add to the end of the queue
8 while  $q$  is not empty do
9    $b \leftarrow q.\text{dequeue}()$ ; // remove from the front of the queue
10   $q.\text{enqueue}(b.\text{children})$ ; // break ties with any heuristics
11  if  $b.\text{type} = \text{leaf}$  then
12     $\mathcal{I}(b) \leftarrow \{\}$ ; // if  $b$  is a leaf bag
13  else
14     $\mathcal{I}(b) \leftarrow \{\varphi \in C \mid \text{var}(\varphi) \subseteq b\}$ ; // if  $b$  is of other types
15     $C \leftarrow C \setminus \mathcal{I}(b)$ ;
16  end
17 end
18  $\triangleright$  Recursively compute  $\mathcal{V}$  from bottom up.
19  $q.\text{enqueue}(\mathcal{B}.\text{leaves})$ ;
20 while  $q$  is not empty do
21    $b \leftarrow q.\text{dequeue}()$ ;
22    $q.\text{enqueue}(b.\text{parent})$ ;
23   Compute  $\mathcal{V}(b)$  according to Eq. (3);
24 end
25 return  $\bigwedge \mathcal{V}(r_T)$ 

```

of the heuristics impacts the overall efficiency in the practical implementation, which will be detailed in Sect. 5.

Step 3: Recursion (Line 18–25). The value function $\mathcal{V}$ is then updated from bottom up, i.e., $\mathcal{V}(b)$ is updated after the values for all b 's children have been updated. Let us denote by $b.\text{(left/right)child}$ the (left/right) child of a (join) bag b , by $b.\text{children}$ the collection of all children of a bag b , and by $b.\text{forget}$ the removed variable at a forget bag b . At each step, $\mathcal{V}(b)$ is updated according to the type of the bag $b \in T$:

$$\mathcal{V}(b) \leftarrow \begin{cases} \mathcal{I}(b) \cup \mathcal{V}(b.\text{leftchild}) \cup \mathcal{V}(b.\text{rightchild}) & \text{if } b \text{ is a join bag} \\ \mathcal{I}(b) \cup \mathcal{V}(b.\text{children}) & \text{if } b \text{ is an introduce bag} \\ \mathcal{I}(b) \cup \text{FME}(\bigwedge \mathcal{V}(b.\text{children}), b.\text{forget}) & \text{if } b \text{ is a forget bag} \\ \mathcal{I}(b) & \text{if } b \text{ is a leaf bag} \end{cases} \quad (3)$$

Finally, the algorithm outputs the conjunction of all atomic formulas stored in $\mathcal{V}(r_T)$.

Theorem 1 (Correctness). *The formula $\bigwedge \mathcal{V}(r_T)$ is equivalent to Φ .*

Proof. Let $T_b = (\mathcal{B}_b, E_{T_b})$ denote the sub-tree rooted at a bag b . We denote by $\text{ElimVar}(T_b) \triangleq (\bigcup_{b' \in T_b} b') \setminus b$ the set of removed variables at forget bags in T_b . In this proof, we prove a stronger result by showing that, for each bag $b \in \mathcal{B}$, $\bigwedge \mathcal{V}(b) \equiv \exists \text{ElimVar}(T_b). \bigwedge_{b' \in \mathcal{B}_b} \mathcal{I}(b')$. Therefore, the original statement is a special case when b is the root r of T , by noting that $\Phi = \exists x_m, \dots, \exists x_1. \bigwedge_{b \in \mathcal{B}} \mathcal{I}(b)$.

The proof is done by induction on the structure of the nice tree decomposition T . Suppose the above property holds for every descendant of a bag b , we shall prove that the property still holds for b , depending on the type of b . When b is a join, introduce, or leaf bag, the proof is straightforward by the definition of $\mathcal{V}$. When b is a forget bag, we have

$$\begin{aligned}
\bigwedge \mathcal{V}(b) &= \bigwedge (\mathcal{I}(b) \cup \text{FME}(\bigwedge \mathcal{V}(b.\text{children}), b.\text{forget})) \\
&\equiv \bigwedge \mathcal{I}(b) \wedge \bigwedge \text{FME}(\exists \text{ElimVar}(T_{b.\text{children}}). \bigwedge_{b' \in \mathcal{B}_{b.\text{children}}} \mathcal{I}(b'), b.\text{forget}) \\
&\equiv \bigwedge \mathcal{I}(b) \wedge \left(\exists \text{ElimVar}(T_{b.\text{children}}) \exists b.\text{forget}. \bigwedge_{b' \in \mathcal{B}_{b.\text{children}}} \mathcal{I}(b') \right) \\
&\equiv \exists \text{ElimVar}(T_b). \bigwedge \mathcal{I}(b) \wedge \bigwedge_{b' \in \mathcal{B}_{b.\text{children}}} \mathcal{I}(b') \equiv \exists \text{ElimVar}(T_b). \bigwedge_{b' \in \mathcal{B}_b} \mathcal{I}(b'),
\end{aligned}$$

where the second line is obtained by applying the induction hypothesis, and the last line is correct because variables in $\text{ElimVar}(T_b)$ do not occur in $\mathcal{I}(b)$. $\square$

3.2 Extending to CAD and general QE

Extending to CAD's projection phase. The framework Alg. 1 can be utilized in CAD's projection phase with only minor modifications. We present the whole algorithm in Appx. A.3 and here only explain the differences: (i) The two value functions $\mathcal{I}$ and $\mathcal{V}$ are defined as mappings from bags in T to set of polynomials in $\mathbb{R}[x_1, \dots, x_n]$, instead of atomic formulas; (ii) In the recursion step, for a forget bag b , we compute $\mathcal{V}(b)$ via $\mathcal{V}(b) \leftarrow \mathcal{I}(b) \cup \text{Proj}(\mathcal{V}(b.\text{children}), b.\text{forget})$, while for other types of bags, the computation of $\mathcal{V}(b)$ remains the same as in Alg. 1; and (iii) Finally, after applying the projection operation for all quantified variables, the CAD algorithm continues the projection phases for free variables, after which the lifting phase can start. The correctness of the framework follows from the fact that the algebraic operations in the projection operator (presented in Appx. A.1) preserve locality, i.e., will not introduce new variables.

We shall note here that we *cannot* simply replace the invocation of the FME algorithm in Alg. 1 by the CAD algorithm, i.e., the projection phase and lifting phase of CAD must be separated. This is because the lifting phase produces a formula in disjunction normal form, on which we can not directly apply the dynamic programming framework.

Extending to general quantifier elimination algorithms. We now extend our framework to an arbitrary logic theory $\mathcal{T}$ that admits quantifier elimination. This is achieved by extracting a variable elimination order from the tree decomposition, which involves the following steps: (1) Construct a nice tree decomposition T of the input $\mathcal{T}$ -formula's primal graph; (2) Traverse T from top to bottom to obtain an ordering of the eliminated variables corresponding to the forget bags; and (3) Finally, reverse the ordering and use it as the variable elimination order in any quantifier elimination procedure for theory $\mathcal{T}$. The pseudo-code description of the algorithm is presented in Appx. A.3. The correctness of the algorithm is straightforward, as the change of variable order will not affect the correctness of a quantifier elimination algorithm.

In fact, the variable ordering extracted via the above algorithm can be shown to be a *perfect elimination ordering*¹, say (1, 3, 8, 7, 6, 5, 4, 2) for our running example. Hence, it is possible that our variable ordering coincides with the one extracted from a chordal extension graph [53]. However, since we have exploited the sparsity information in building the tree decompositions, our variable orderings are usually superior for performing quantifier elimination on sparse problem instances. This will be further evidenced by our experimental results in Sect. 5. Moreover, our framework is stated for general quantifier elimination procedures and enables a fine-grained complexity analysis, explained in the following Sect. 4.

4 Complexity Analysis

In this section, we analyze the worst-case complexity of our framework in Sect. 3. The core idea is to combine the standard analysis with a special kind of tree decomposition, called *balanced tree-decomposition*. We prove that when the treewidth is constant, a nice and balanced tree-decomposition with logarithmic height in the number of variables can be found in sub-polynomial time. By working on this tree decomposition, our dynamic programming framework can achieve exponential improvement in the worst-case complexity for both FME and CAD. To our knowledge, this gives the first complexity result of FME and CAD directly related to treewidth sparsity.

Balanced tree-decomposition. The concept of (β, γ) -balanced tree decompositions [17] generalizes the notion of balanced tree decompositions that arise in the analysis of the space complexity of tree decomposition algorithms [9, 30]. Informally, a (β, γ) -balanced tree-decomposition requires, for each bag b , the size of subtree T_b decreases proportionally as $Lv(b)$ increases.

Definition 1 ((β, γ)-Balanced Tree Decomposition). For constants $0 < \beta < 1$ and $\gamma \in \mathbb{N}_{>0}$, a binary tree-decomposition T is called (β, γ) -balanced if, for every bag b , every descendant b' of b with $Lv(b') - Lv(b) = \gamma$ satisfies that $|T_{b'}| \leq \beta \cdot |T_b|$.

¹ In graph theory, a perfect elimination ordering of a graph G is an ordering of its vertices such that for every vertex v , the set consisting of v and its neighbors that come after v in the ordering forms a clique in G .

The construction of a (β, γ) -balanced tree decomposition is elucidated in [17, Sec. 3], which gives the following theorem.

Theorem 2 ([17, Thm. 3.1]). *Given a graph G with n vertices and of constant treewidth k . For any fixed $\delta > 0$ and $\lambda \in \mathbb{N}$ with $\lambda \geq 2$, a binary (β, γ) -balanced tree-decomposition T with $\beta = (\frac{1+\delta}{2})^{\lambda-1}$ and $\gamma = \lambda$ can be constructed in $O(n \cdot \log n)$ time and $O(n)$ space. Moreover, the tree decomposition T has $O(n)$ bags and width $\frac{6\lambda}{\delta} \cdot (k+1) - 1$ at most.*

Relying on the above theorem, we can construct a nice tree decomposition with logarithmic height, as stated below.

Proposition 1. *Given a graph G with n vertices and of constant treewidth k , a nice tree decomposition T of height $O(\log n)$ and constant width can be constructed in $O(n \log n)$ time.*

Proof. Take $\delta = 1/2$ and $\lambda = 2$ in Thm. 2, then a $(3/4, 2)$ -balanced tree decomposition $T = (\mathcal{B}, E_T)$ of width at most $24(k+1) - 1$ can be constructed in $O(n \cdot \log n)$ time. Because of the $(3/4, 2)$ -balancing, we have in every 2-levels of T a decrease of at least $3/4$ in the size of the bags. Therefore, the height of T is $2 \log_{3/4} n$, which is $O(\log n)$. Then, we extend T into a nice tree decomposition, which involves the following steps:

Step 1: reroot. Choose an arbitrary bag $r \in \mathcal{B}$ of T to be its current root. We first construct a new bag r' such that $V_{r'} = \emptyset$ as the new root of the nice tree decomposition. Then, a sequence of forget bags is introduced to remove the variables in V_r one by one until reaching r' . This step takes $O(k)$ time and increases the height of T by k at most.

Step 2: ensure join bags have identical children. For each bag b that has two children, we construct two more bags that are identical to b and insert them between b and its two children, respectively. This step takes $O(n)$ time and will double the height at most.

Step 3: create introduce and forget chains. After the previous steps, for each edge (b_1, b_2) in T , the bags V_{b_1} and V_{b_2} may still differ by more than one variables. So we insert between b_1 and b_2 a chain of introduce and forget bags to make sure that every consecutive pair of bags differ in at most one variable. This step takes $O(kn)$ time and increases the height by up to k times at most.

To conclude, when k is a constant, we can obtain a nice tree decomposition of height $O(\log n)$ in $O(n)$ time. Hence, the total time cost remains $O(n \log n)$ as in Thm. 2. $\square$

In the following, we analyze the complexity of our frameworks for FME and CAD in Sect. 3 under the assumption that k is a constant. For consistency², we assume that $m = n$ in the input formula Eq. (1), i.e., there are no free variables.

Complexity analysis for FME. The complexity of the FME algorithm can be measured by the number of atomic formulas produced in the elimination process.

² Recall that the CAD's projection phrase requires performing the projection operation for all variables, including the quantifier-free ones.

Assume that there are s atomic formulas in the input formula Eq. (1). Then, after eliminating the variable x_r , the number of atomic formulas in the output depends on the number of lower/upper bounds, which is at most $|L_r| \cdot |U_r| \leq \frac{s^2}{4}$. Therefore, the complexity of eliminating n variables is at most $O(s^{2^n})$.

By combining the standard analysis with the nice tree decomposition obtained in Prop. 1, we show that a single exponential complexity upper bound w.r.t. n can be achieved when the treewidth is a constant.

Theorem 3. *If the primal graph of an LRA formula Φ in Eq. (1) has constant treewidth and includes s atomic formulas, the number of atomic formulas in the output of Alg. 1 is $s^{O(n)}$ at most.*

Proof. Let T be the nice tree decomposition given by Prop. 1 with height h . In the following, we count the atomic formulas stored in $\mathcal{V}(b)$ for each bag b in T . For a bag b with $\text{Lv}(b) = h$, which must be a leaf bag, we have $\mathcal{V}(b) = \mathcal{I}(b) \leq s$. Consider going one step up, for a bag b with $\text{Lv}(b) = h - 1$: if b is a join, introduce, or leaf bag, we have $\mathcal{V}(b) \leq 3s$ according to the definition of $\mathcal{V}$; if b is a forget bag, we apply the one-step analysis for FME and have $\mathcal{V}(b) \leq s + \frac{s^2}{4}$. Since the height h is bounded by $O(\log n)$ according to Prop. 1, for the root r , we have $\mathcal{V}(r) = O(s^{2^{O(\log n)}}) = s^{O(n)}$. $\square$

Complexity analysis for CAD. The CAD complexity is usually measured by the number of sign-invariant cells. Our analysis uses a similar approach as [53] but provides an improved bound based on the assumption of constant treewidth and the structure of balanced tree decompositions.

Before presenting the results, we introduce some tools to estimate the growth of the size of the polynomials in the projection phase. For a set of polynomials $P \subset \mathbb{R}[\mathbf{x}]$, we define the *combined degree* [58] of P to be $\max_{1 \leq i \leq n} \deg_{x_i}(\prod_{p \in P} p)$, where $\deg_{x_i}(\cdot)$ denotes the degree of a polynomial in a single variable x_i . A set of polynomials has the (m, d) -property if it can be partitioned into m sets, such that each set has maximum combined degree d [10, Def. 7]. It has been shown in [10, Lem. 11] that for a set $P \subset \mathbb{R}[\mathbf{x}]$ with the (m, d) -property, the set $\text{Proj}(P, x_i)$ after projection has the $(M, 2d^2)$ -property with $M = \lfloor \frac{(m+1)^2}{2} \rfloor$ for any $x_i \in \mathbf{x}$.

Proposition 2. *If the primal graph of an NRA formula Φ in Eq. (1) has constant treewidth and the corresponding polynomial set has the (m, d) -property, the number of cells in the CAD algorithm can be made $m^{O(n)}(2Md)^{O(n \log n)}$ at most.*

Proof. The proof is inspired by the proof of [53, Thm. 11, Thm. 12]. Here we explain the main idea.

We first analyze the increase in the number of polynomials in the projection phase. Let T be the nice tree decomposition given by Prop. 1 with height h . By assumption, for each leaf bag b with $\text{Lv}(b) = h$, the polynomial set $\mathcal{V}(b)$ has the (m, d) -property. Following a similar argument in the proof of Thm. 3, we can show that for each bag b with $\text{Lv}(b) \leq h - 1$, the polynomial set $\mathcal{V}(b)$ has the $(3^{2^{l_b}-1} M^{2^{l_b}-1}, 2^{2^{l_b}-1} d^{2^{l_b}})$ -property with $l_b = h - 1 - \text{Lv}(b)$.

Then we analyze the lifting phase. Note that, for a polynomial set P , the real roots of the product $\prod_{p \in P} p$ include all real roots of each individual polynomial in P . Hence, if a univariate polynomial set P has the (m, d) -property, the number of real roots in P is at most md , and the number of the corresponding sets in $\mathbb{R}^1$ is at most $2md + 1$. The total number of cells produced in the lifting phase is bounded by the product of all $\mathbb{R}^1$ -cells corresponding to each bag b , i.e. $\prod_{b \in \mathcal{B}} (2K_b + 1)$, where

$$K_b = \begin{cases} md, & \text{if } \text{Lv}(b) = h, \text{ i.e., } l_b \text{ undefined;} \\ 3^{2^{l_b}-1} M^{2^{l_b}-1} 2^{2^{l_b}-1} d^{2^{l_b}}, & \text{if } h-1 \geq \text{Lv}(b) \geq 0, \text{ i.e., } 0 \leq l_b \leq h-1. \end{cases}$$

Since T is a binary tree, the number of bags on each level is bounded by $2^{\text{Lv}(b)}$. Therefore, we have

$$\begin{aligned} \prod_{b \in \mathcal{B}} (2K_b + 1) &\leq (md)^{2^h} \prod_{0 \leq l_b \leq h-1} (3^{2^{l_b}-1} M^{2^{l_b}-1} 2^{2^{l_b}-1} d^{2^{l_b}})^{2^{h-1-l_b}} \\ &\leq m^{2^h} 6^{(h-2)2^{h-1}+1} M^{h2^{h-2}} d^{(h+2)2^{h-1}} = O(m^{2^h} 6^{h2^{h-1}} M^{h2^{h-2}} d^{h2^{h-1}}) \\ &= m^{O(n)} (2Md)^{O(n \log n)} \end{aligned}$$

where the final step is obtained by using $h = O(\log n)$. $\square$

5 Implementation and Experiments

In this section, we explain our implementation details and present the experimental results. We aim to answer the following research question: Compared with the existing heuristics for deciding the variable elimination ordering, *how does our tree-decomposition-based algorithm perform on the QE problems of LRA and NRA instances with relatively small treewidth?*

Computation Environment. We use the tool from [71] to compute tree decompositions, which are then processed by Python scripts. For LRA, we compare different heuristics based on a naive FME implementation in Python, while for NRA, we work on CAD implemented in MATHEMATICA [45]. All experiments were conducted on a Windows PC with i7-13700 CPU and 32GB RAM.

Existing Heuristics. For comparison, we briefly explain the most frequently used heuristic strategies for selecting the variable elimination ordering in FME and CAD. For FME, the baseline Random strategy makes $N = 5$ trials according to different orderings that are sampled randomly and records the best result; the Greedy strategy chooses the next variable to be eliminated by minimizing the number of generated constraints. For CAD, the Brown's heuristic, denoted Brown [14], chooses the next eliminated variable that has the lowest maximum degree across all polynomials in which it appears (and breaks ties with addition rules); the PEO strategy [53] extracts a perfect elimination ordering from the chordal extension of the primal graph; and we also compare with the default strategy in MAPLE [56] to suggest variable ordering, denoted SVO.

Table 1: FME performance on randomly generated LRA instances

ID	Instance Set (10 instances per set)				Random	Greedy	Ours
1	#var=15 #ineq=75 Ave. ineq. count				51,608.4	4,404.7	4,841.3
	#elim=5 #tw=2 Ave. run Time(s)				0.21	0.018	0.018
2	#var=15 #ineq=75 Ave. ineq. count				>10,000,000	28,362.3	28,244.8
	#elim=5 #tw=4 Ave. run Time(s)				NA	0.17	0.17
3	#var=20 #ineq=100 Ave. ineq. count				>10,000,000	290,597.8	194,645.3
	#elim=10 #tw=2 Ave. run Time(s)				NA	2.0	1.1
4	#var=20 #ineq=100 Ave. ineq. count				>10,000,000	313,584.0	313,585.9
	#elim=6 #tw=4 Ave. run Time(s)				NA	2.6	2.4
5	#var=30 #ineq=150 Ave. ineq. count				>10,000,000	343,705.4	110,304.9
	#elim=12 #tw=3 Ave. run Time(s)				NA	3.7	1.2
6	#var=30 #ineq=150 Ave. ineq. count				>10,000,000	974,233.4	974,233.4
	#elim=5 #tw=8 Ave. run Time(s)				NA	14.9	14.4

Our Implementation. In our practical implementation, we elected not to construct a balanced tree decomposition as described in Sect. 4. While the main advantage of the balanced tree decomposition lies in estimating the asymptotic complexity, it often performs suboptimally on problems of relatively small scale. Instead, we work with the tree decomposition computed by [71] and follow the approach detailed in Sect. 3.2 to extract the elimination order³. When traversing from top to bottom, we break ties using the aforementioned Greedy and Brown heuristics for FME and CAD, respectively.

Benchmarks. As our algorithm targets dealing with LRA and NRA problems that are *small enough* to do quantifier elimination but also *have low treewidth*. This requirement rules out most existing standard benchmarks (an analysis on SMT-LIB benchmarks [5] can be found in Appx. A.4). Hence, for comparison, we generated a set of benchmarks with treewidth sparsity as follows: We construct benchmark inequality systems from graphs of prescribed treewidth k by a two-stage procedure. We first generate a graph of treewidth k by iteratively attaching nodes to a $(k + 1)$ -clique. Then, for each bag, we enumerate all monomials of total degree not exceeding the maximum degree (1 for LRA), select at least one highest-degree monomial, and randomly include other monomials with a probability ranging from 0.05 to 0.15, while ensuring that every scoped variable appears. For each set of hyperparameters, we randomly generated 10 examples for LRA and 1 example for NRA, because the randomly selected monomials significantly impact the CAD efficiency, and the average runtime is usually dominated by the worst instance.

Results. Table 1 and Table 2 present the results of FME and CAD, respectively. On LRA benchmarks, our strategy consistently yielded the lowest average inequality count and average runtime, achieving the best result in 4 out of 6 sets.

³ In our non-parallel setting, using the elimination order is functionally equivalent to running the dynamic programming framework, without the explicit computation of the value functions.

Table 2: CAD performance on randomly generated NRA instances

ID	Instance	SVO	Brown	PEO	Ours
7	#var=6 #ineq=6 Cells	1,133,532	271,120	274,724	110,828
	#max_deg=2 #tw=2 Time(s)	640.3	307.0	114.7	40.5
8	#var=7 #ineq=4 Cells	126,328	89,804	89,804	49,108
	#max_deg=3 #tw=3 Time(s)	42.1	34.5	29.0	11.6
9	#var=7 #ineq=5 Cells	2,020,378	1,926,208	740,380	181,090
	#max_deg=2 #tw=2 Time(s)	702.0	416.2	96.1	84.1
10	#var=8 #ineq=5 Cells	3,721,372	3,721,372	984,008	2,155,132
	#max_deg=4 #tw=2 Time(s)	1843.8	1718.2	567.7	1090.3
11	#var=8 #ineq=6 Cells	38,212	69,748	69,748	38,212
	#max_deg=2 #tw=3 Time(s)	42.0	633.3	427.9	42.0
12	#var=9 #ineq=10 Cells	1,804,224	1,804,224	954,432	52,992
	#max_deg=2 #tw=3 Time(s)	742.3	746.7	156.7	83.3

The Greedy strategy showed a very similar performance with the runtime difference being negligible, but was significantly slower on ID 3 and 5. The Random strategy performed poorly, generating an extremely high number of inequalities, often exceeding 10^7 for most instances. On NRA benchmarks, our strategy also demonstrates superior performance in all but one benchmark, in terms of both the number of cells generated and the runtime. For benchmark ID 9 and ID 12, it significantly outperforms all other methods. The only instance where our strategy did not have the lowest cell count was ID 10, where PEO was superior. The results answer our research question affirmatively, indicating that *our proposed strategy is effective and more efficient in dealing with problems with low treewidth*, compared to the existing heuristics. More experiments on benchmarks from [53] are available at Appx. A.5.

6 Conclusion

This paper introduces a high-level, treewidth-aware approach to reduce the computational burden of quantifier elimination procedures, especially FME and CAD, for problems with treewidth sparsity. By leveraging parameterized algorithm tools, our method establishes an improved complexity upper bound, thus offering novel insights into mitigating the inherent complexity of quantifier elimination. Future work will focus on combining this treewidth-aware method with existing heuristics and exploring its application in various practical scenarios.

Acknowledgments. We thank the anonymous reviewers for their valuable comments and helpful suggestions. This work has been partially funded by the National Key R&D Program of China under grant No. 2022YFA1005101 and 2022YFA1005102, the National NSF of China under grant No.62192732 and W2511064, the ERC Starting Grant 101222524 (SPES), and the Ethereum Foundation Research Grant FY24-1793.

Data Availability Statement. The code for our experiments is available at <https://github.com/Jiyu-ISCAS/qe-treewidth-artifact>.

References

1. An, J., Zhan, N., Li, X., Zhang, M., Yi, W.: Model checking bounded continuous-time extended linear duration invariants. In: HSCC. pp. 81–90 (2018)
2. Anai, H., Weispfenning, V.: Reach set computations using real quantifier elimination. In: HSCC. pp. 63–76 (2001)
3. Arnborg, S., Corneil, D.G., Proskurowski, A.: Complexity of finding embeddings in a k-tree. *SIAM J. Algebraic Discrete Methods* **8**(2), 277–284 (1987)
4. Arnon, D.S., Collins, G.E., McCallum, S.: Cylindrical algebraic decomposition I: the basic algorithm. *SIAM J. Comput.* **13**(4), 865–877 (1984)
5. Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org (2016)
6. Bodlaender, H.L.: Dynamic programming on graphs with bounded treewidth. In: ICALP. pp. 105–118 (1988)
7. Bodlaender, H.L.: A linear-time algorithm for finding tree-decompositions of small treewidth. *SIAM J. Comput.* **25**(6), 1305–1317 (1996)
8. Bodlaender, H.L.: Treewidth: Algorithmic techniques and results. In: MFCS. pp. 19–36 (1997)
9. Bodlaender, H.L., Hagerup, T.: Parallel algorithms with optimal speedup for bounded treewidth. *SIAM J. Comput.* **27**(6), 1725–1746 (1998)
10. Bradford, R.J., Davenport, J.H., England, M., McCallum, S., Wilson, D.J.: Truth table invariant cylindrical algebraic decomposition. *J. Symb. Comput.* **76**, 1–35 (2016)
11. Bradford, R.J., Davenport, J.H., England, M., Wilson, D.J.: Optimising problem formulation for cylindrical algebraic decomposition. In: CICM. pp. 19–34 (2013)
12. Bradley, A.R., Manna, Z.: The calculus of computation - decision procedures with applications to verification. Springer (2007)
13. Brown, C.W.: Improved projection for cylindrical algebraic decomposition. *J. Symb. Comput.* **32**(5), 447–465 (2001)
14. Brown, C.W.: Companion to the tutorial presented at issac’04: Cylindrical algebraic decomposition. In: ISSAC, tutorial (2004), <https://www.usna.edu/Users/cs/wcbrown/research/ISSAC04/handout.pdf>
15. Brown, C.W.: Open non-uniform cylindrical algebraic decompositions. In: ISSAC. pp. 85–92 (2015)
16. Caviness, B.F., Johnson, J.R.: Quantifier Elimination and Cylindrical Algebraic Decomposition. Texts and Monographs in Symbolic Computation (1998)
17. Chatterjee, K., Ibsen-Jensen, R., Goharshady, A.K., Pavlogiannis, A.: Algorithms for algebraic path properties in concurrent systems of constant treewidth components. *ACM Trans. Program. Lang. Syst.* **40**(3), 9:1–9:43 (2018)
18. Chen, C., Maza, M.M., Xia, B., Yang, L.: Computing cylindrical algebraic decomposition via triangular decomposition. In: ISSAC. pp. 95–102 (2009)
19. Chen, C., Zhu, Z., Chi, H.: Variable ordering selection for cylindrical algebraic decomposition with artificial neural networks. In: International Congress on Mathematical Software. pp. 281–291 (2020)
20. Collins, G.E.: Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In: Automata Theory and Formal Languages, 2nd GI Conference. pp. 134–183. Springer (1975)
21. Collins, G.E., Hong, H.: Partial cylindrical algebraic decomposition for quantifier elimination. *J. Symb. Comput.* **12**(3), 299–328 (1991)

22. Cygan, M., Fomin, F.V., Kowalik, L., Lokshtanov, D., Marx, D., Pilipczuk, M., Pilipczuk, M., Saurabh, S.: Parameterized algorithms. Springer (2015)
23. Davenport, J.H., Heintz, J.: Real quantifier elimination is doubly exponential. *Journal of Symbolic Computation* **5**(1-2), 29–35 (1988)
24. Dell, H., Komusiewicz, C., Talmon, N., Weller, M.: The PACE 2017 Parameterized Algorithms and Computational Experiments Challenge: The Second Iteration. In: IPEC (2017)
25. Diestel, R.: Graph Theory (2nd ed.). Springer (2000)
26. Dolzmann, A., Seidl, A., Sturm, T.: Efficient projection orders for CAD. In: ISSAC. pp. 111–118 (2004)
27. Dong, S., Lee, Y.T., Ye, G.: A nearly-linear time algorithm for linear programs with small treewidth: a multiscale representation of robust central path. In: STOC. pp. 1784–1797 (2021)
28. Dorato, P., Yang, W., Abdallah, C.T.: Robust multi-objective feedback design by quantifier elimination. *J. Symb. Comput.* **24**(2), 153–159 (1997)
29. Downey, R.G., Fellows, M.R.: Parameterized complexity. Springer (2012)
30. Elberfeld, M., Jakobý, A., Tantau, T.: Logspace versions of the theorems of bodlaender and courcelle. In: FOCS. pp. 143–152 (2010)
31. England, M., Florescu, D.: Comparing machine learning models to choose the variable ordering for cylindrical algebraic decomposition. In: CICM. pp. 93–108 (2019)
32. Fichte, J.K., Hecher, M., Kieler, M.F.I.: Treewidth-aware quantifier elimination and expansion for QCSP. In: CP. pp. 248–266 (2020)
33. Fourier, J.B.J.: Analyse des travaux de l’Academie Royale des Sciences, pendant l’année 1827. Partie mathématique (1826)
34. Gan, T., Chen, M., Li, Y., Xia, B., Zhan, N.: Reachability analysis for solvable dynamical systems. *IEEE Trans. Autom. Control.* **63**(7), 2003–2018 (2018)
35. Gu, Y., Song, Z.: A faster small treewidth sdp solver (2022), <https://arxiv.org/abs/2211.06033>
36. Halin, R.: S-functions for graphs. *Journal of geometry* **8**, 171–186 (1976)
37. Han, J., Dai, L., Hong, H., Xia, B.: Open weak CAD and its applications. *J. Symb. Comput.* **80**, 785–816 (2017)
38. Han, J., Jin, Z., Xia, B.: Proving inequalities and solving global optimization problems via simplified CAD projection. *J. Symb. Comput.* **72**, 206–230 (2016)
39. Hong, H.: An improvement of the projection operator in cylindrical algebraic decomposition. In: ISSAC. pp. 261–264 (1990)
40. Hong, H., Din, M.S.E.: Variant quantifier elimination. *J. Symb. Comput.* **47**(7), 883–901 (2012)
41. Hong, H., Liska, R., Steinberg, S.L.: Testing stability by quantifier elimination. *J. Symb. Comput.* **24**(2), 161–187 (1997)
42. Huang, Z., England, M., Wilson, D.J., Davenport, J.H., Paulson, L.C., Bridge, J.P.: Applying machine learning to the problem of choosing a heuristic to select the variable ordering for cylindrical algebraic decomposition. In: CICM. pp. 92–107 (2014)
43. Imbert, J.: About redundant inequalities generated by fourier’s algorithm. In: AIMS. pp. 117–127 (1990)
44. Imbert, J.: Fourier’s elimination: Which to choose? In: PPCP. pp. 117–129 (1993)
45. Inc., W.R.: Mathematica, Version 14.3, <https://www.wolfram.com/mathematica>, champaign, IL, 2025

46. Jia, F., Dong, Y., Liu, M., Huang, P., Ma, F., Zhang, J.: Suggesting variable order for cylindrical algebraic decomposition via reinforcement learning. *NeurIPS* **36**, 76098–76119 (2023)
47. Jing, R., Maza, M.M., Talaashrafi, D.: Complexity estimates for fourier-motzkin elimination. In: *CASC*. pp. 282–306 (2020)
48. Jirstrand, M.: Cylindrical algebraic decomposition-an introduction. Technical Report, Linköping University (1995)
49. Jirstrand, M.: Nonlinear control system design by quantifier elimination. *J. Symb. Comput.* **24**(2), 137–152 (1997)
50. Kapur, D.: A quantifier-elimination based heuristic for automatically generating inductive assertions for programs. *Journal of Systems Science and Complexity* **19**, 307–330 (2006)
51. Kloks, T.: *Treewidth, Computations and Approximations*. Springer (1994)
52. Kroening, D., Strichman, O.: *Decision Procedures: An Algorithmic Point of View*. Springer Publishing Company, Incorporated (2008)
53. Li, H., Xia, B., Zhang, H., Zheng, T.: Choosing better variable orderings for cylindrical algebraic decomposition via exploiting chordal structure. *J. Symb. Comput.* **116**, 324–344 (2023)
54. Liu, J., Zhan, N., Zhao, H.: Computing semi-algebraic invariants for polynomial dynamical systems. *EMSOFT* pp. 97–106 (2011)
55. Mallach, S.: On integer linear programs for treewidth based on perfect elimination orderings (extended version). *Acta Informatica* **62**(3), 34 (2025)
56. Maplesoft, a division of Waterloo Maple Inc.: Maple, <https://hadoop.apache.org, waterloo, Ontario>, 2019
57. Marx, D.: Can you beat treewidth? *Theory Comput.* **6**(1), 85–112 (2010)
58. McCallum, S.: An improved projection operation for cylindrical algebraic decomposition. Phd thesis (computer sciences tech. rep. 578), Univ. Wisconsin–Madison. (1985)
59. McCallum, S.: An improved projection operation for cylindrical algebraic decomposition of three-dimensional space. *J. Symb. Comput.* **5**(1/2), 141–161 (1988)
60. Motzkin, T.S.: *Beiträge zur theorie der linearen ungleichungen*. Azriel (1936)
61. Promies, V., Ábrahám, E.: A divide-and-conquer approach to variable elimination in linear real arithmetic. In: *FM* (1). pp. 131–148 (2024)
62. Promies, V., Nalbach, J., Ábrahám, E., Kobialka, P.: Fmplex: Exploring a bridge between fourier-motzkin and simplex. *Log. Methods Comput. Sci.* **21**(2) (2025)
63. Robertson, N., Seymour, P.D.: Graph minors. iii. planar tree-width. *Journal of Combinatorial Theory, Series B* **36**(1), 49–64 (1984)
64. Robertson, N., Seymour, P.D.: Graph minors. II. algorithmic aspects of tree-width. *J. Algorithms* **7**(3), 309–322 (1986)
65. Rodríguez-carbonell, E., Kapur, D.: Automatic generation of polynomial loop invariants: Algebraic foundations. In: *ISSAC*. pp. 266–273 (2004)
66. Saugata Basu, Richard Pollack, M.F.R.: *Algorithms in Real Algebraic Geometry*. Springer Berlin, Heidelberg (2006)
67. Schrijver, A.: *Theory of linear and integer programming*. John Wiley & Sons (1998)
68. Strzebonski, A.W.: Cylindrical algebraic decomposition using local projections. *J. Symb. Comput.* **76**, 36–64 (2016)
69. Sturm, T., Tiwari, A.: Verification and synthesis using real quantifier elimination. In: *ISSAC*. pp. 329–336. ACM (2011)
70. Szeider, S.: Finding paths in graphs avoiding forbidden transitions. *Discret. Appl. Math.* **126**(2-3), 261–273 (2003)

71. Tamaki, H.: Positive-instance driven dynamic programming for treewidth. *J. Comb. Optim.* **37**(4), 1283–1311 (2019)
72. Tarski, A.: The Completeness of Elementary Algebra and Geometry. Reprinted in 1967 by CNRS, Institute Blaise Pascal, Paris (1930)

A Appendix

A.1 CAD Algorithm

In this part, we provide a formal description of the CAD algorithm. The notations and definitions mostly follow Jirstrand’s introductory paper [48]. Given a set of polynomials in $\mathcal{F} \subset \mathbb{R}[x_1, \dots, x_n]$, the (CAD) algorithm decomposes the $\mathbb{R}^n$ space into finitely many sign-invariant regions, called *cells*. After decomposition, one can determine the sign of the polynomials in each cell using sampled points in this cell.

In the one-dimensional case, the CAD algorithm can be understood as computing all the real roots of a set of univariate polynomials. Then, the sign-invariant cells are exactly the real roots themselves, along with the intervals they divide. Using this idea, n -dimensional CAD is constructed recursively based on $(n - 1)$ -dimensional CAD.

We first set some basic definitions. A region R is a connected subset of $\mathbb{R}^n$. The set $Z(R) = R \times \mathbb{R}$ is called a *cylinder* over R . Let f, f_1, f_2 be continuous, real-valued functions over R . A f -section of $Z(R)$ is the set $\{(x, f(x)) \mid x \in R\}$. A (f_1, f_2) -sector of $Z(R)$ is the set $\{(x, y) \mid x \in R, f_1(x) < y < f_2(x)\}$. A decomposition of X is a finite collection of disjoint regions components whose union is X .

Definition 2 (Stack). *A stack over R is a decomposition which consists of f_i -sections and (f_i, f_{i+1}) -sectors, where $f_0 < \dots < f_{k+1}$ for all $x \in R$ and $f_0 = -\infty, f_{k+1} = +\infty$.*

Definition 3 (Delineable). *A set of polynomials is delineable over a region R if the number of distinct collective roots remains a constant, i.e., their roots do not intersect or disappear.*

Definition 4 (Cylindrical Decomposition). *A decomposition $\mathcal{D}$ of $\mathbb{R}^n$ is cylindrical if: (i) when $n = 1$, $\mathcal{D}$ is a partition of $\mathbb{R}$ into finitely set of numbers, and the finite and infinite open intervals bounded by these numbers. (ii) when $n > 1$, there exists a cylindrical decomposition $\mathcal{D}'$ of $\mathbb{R}^{n-1}$, and over each region $R \in \mathcal{D}'$, there is a stack belonging to $\mathcal{D}$.*

Definition 5 (Cylindrical Algebraic Decomposition). *A decomposition is algebraic if each of its components is a semi-algebraic set. A Cylindrical Algebraic Decomposition (CAD) of $\mathbb{R}^n$ is a decomposition which is both cylindrical and algebraic. The components of a CAD are called cells.*

Defining the projection operator. In the following, we first recall the definition of resultants and discriminants for multivariate polynomials, then give the formal definition of McCallum's projection operator. Let $\mathbb{Z}[x_1, \dots, x_n]$ denote the ring of all integral polynomials in $x_1, \dots, x_n$. For any x_k with $1 \leq k \leq n$, a polynomial $f \in \mathbb{Z}[x_1, \dots, x_n]$ can be written as $f = \sum_{i=0}^d a_i x_k^i$, where the coefficients a_i are polynomials in $\mathbb{R}[x_1, \dots, x_{k-1}, x_{k+1}, \dots, x_n]$ and a_d is not the zero polynomial. The number $d \in \mathbb{N}$ is called the *degree* of f in variable x_k , denoted $\deg(f, x_k)$. The set of non-zero coefficients is denoted by $\text{coeff}(f, x_k)$. The GCD of the coefficients is called the *content* of f w.r.t. x_k denoted by $\text{cont}(f, x_k)$. The polynomial f is called *primitive* w.r.t. x_k if $\text{cont}(f, x_k) = 1$, and the polynomial $f/\text{cont}(f, x_k)$ is called the *primitive part* of f w.r.t. x_k .

Definition 6 (Resultant). Let f_1, f_2 be two polynomials in $\mathbb{R}[x_1, \dots, x_n]$. For a variable x_k with $1 \leq k \leq n$, we write f_1 and f_2 as polynomials in x_k :

$$\begin{aligned} f_1 &= a_s x_k^s + a_{s-1} x_k^{s-1} + \dots + a_0, \\ f_2 &= b_r x_k^r + b_{r-1} x_k^{r-1} + \dots + b_0. \end{aligned}$$

The resultant of f_1 and f_2 with respect to x_k , denoted $\text{Res}(f_1, f_2, x_k)$, is defined as

$$\text{Res}(f_1, f_2, x_k) \triangleq \begin{vmatrix} a_s & a_{s-1} & \dots & a_0 & & & \\ & a_s & a_{s-1} & \dots & a_0 & & \\ & & \ddots & & & \ddots & \\ & & & a_s & a_{s-1} & \dots & a_0 \\ b_r & b_{r-1} & \dots & b_0 & & & \\ & b_r & b_{r-1} & \dots & b_0 & & \\ & & \ddots & & & \ddots & \\ & & & b_r & b_{r-1} & \dots & b_0 \end{vmatrix}.$$

Definition 7 (Discriminant). Let f be a polynomial in $\mathbb{R}[x_1, \dots, x_n]$. For a variable x_k with $1 \leq k \leq n$, assume that

$$f_1 = a_s x_k^s + a_{s-1} x_k^{s-1} + \dots + a_0,$$

The discriminant of f with respect to x_k , denoted $\text{Dis}(f, x_k)$, is defined as

$$\text{Dis}(f, x_k) \triangleq \frac{(-1)^{\frac{s(s-1)}{2}}}{a_s} \text{Res}\left(f, \frac{\partial f}{\partial x_k}, x_k\right).$$

Definition 8 (McCallum's Projection Operator [59]). Let $P \subset \mathbb{Z}[x_1, \dots, x_n]$ be a finite set of polynomials containing at least two variables. Let B be an irreducible basis of the primitive part of P . The McCallum's projector operator, denoted $\text{Proj}(\cdot, \cdot)$, is defined as

$$\begin{aligned} \text{Proj}(P, x_k) &\triangleq \{ \text{cont}(f, x_k) \mid f \in P \} \\ &\cup \{ \text{coeff}(f, x_k) \mid f \in B \} \\ &\cup \{ \text{dis}(f, x_k) \mid f \in B \} \\ &\cup \{ \text{res}(f, g, x_k) \mid f, g \in B, f \neq g \}. \end{aligned}$$

The projection phase. Fix a variable elimination ordering from x_n to x_1 if fixed (which is the reverse of the ordering used in our Sect. 2). The projection operator Proj is applied recursively on $\mathcal{F}$ for $n - 1$ times.

$$\begin{aligned}\mathcal{F}_n &\hat{=} \mathcal{F} \subset \mathbb{R}[x_1, \dots, x_n] \\ \mathcal{F}_{n-1} &\hat{=} \text{Proj}(\mathcal{F}_n, x_n) \subset \mathbb{R}[x_1, \dots, x_{n-1}] \\ &\vdots \\ \mathcal{F}_1 &\hat{=} \text{Proj}(\mathcal{F}_2, x_2) \subset \mathbb{R}[x_1]\end{aligned}$$

In each step, the zero sets of the polynomials in $\mathcal{F}_{i-1}$ is the projection of “significant points” of the zero set of polynomial in $\mathcal{F}_i$, i.e. self crossings, isolated points, vertical tangent points etc.

The lifting phase. We first deal with the 1-dimensional case. Enumerate the real zeros of the monovariate polynomials in $\mathcal{F}_n$ as

$$-\infty < \xi_1 < \xi_2 < \dots < \xi_s < +\infty,$$

which gives an $\mathcal{F}_n$ -invariant decomposition of $\mathbb{R}$. The purpose of this phase is to isolate the above zeros and find sample points for each component in the decomposition. Note that for an open interval we may choose a rational sample point but for a zero we must store an exact representation of the algebraic number.

Then we lift a $\mathcal{F}_i$ -invariant decomposition $\mathcal{D}'$ of $\mathbb{R}^i$ to an $\mathcal{F}_{i+1}$ -invariant decomposition $\mathcal{D}$ of $\mathbb{R}^{i+1}$. By construction, we know that $\mathcal{F}_{i+1}$ is delineable over each cell in $\mathcal{D}'$. For each cell $C \in \mathcal{D}' \in \mathbb{R}^i$, choose a sample point $(\hat{x}_1, \dots, \hat{x}_i) \in \mathbb{R}^i$ and substitute the variables $(x_1, \dots, x_i)$ in $\mathcal{F}_{i+1}$ by these sampled values. In this way, the problem is again reduced to the 1-dimensional case and can be done similarly.

A.2 The Formula in Running Example

$$\Phi \hat{=} \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8.$$

$$\left(\begin{array}{lll} x_1 + 2x_2 + 3x_3 \leq 20 & \wedge -x_1 + x_2 - 2x_3 \leq 5 & \wedge x_1 - 4x_2 \leq 0 \\ \wedge 3x_1 - 4x_2 - 3x_3 \leq 3 & \wedge 3x_1 + x_3 \leq -3 & \wedge 3x_2 - 2x_3 + 4x_4 \leq -5 \\ \wedge 5x_2 - x_4 - 5x_5 \leq 1 & \wedge -5x_2 - 3x_5 + 3x_8 \leq 2 & \wedge 4x_4 + x_5 + x_6 \leq 1 \\ \wedge 3x_6 + 2x_7 \leq -2 & \wedge -5x_3 \leq -2 & \wedge -x_2 - x_3 \leq 1 \\ \wedge x_3 \leq -1 & \wedge x_5 - 5x_6 \leq -4 & \wedge -5x_4 + 5x_5 + 4x_6 \leq -4 \\ \wedge 4x_2 - 4x_5 \leq -5 & \wedge 5x_2 + 3x_3 + 5x_4 \leq 5 & \wedge -4x_2 + 3x_4 + 3x_5 \leq -1 \\ \wedge 3x_2 \leq -4 & \wedge -x_6 - x_7 \leq 2 & \end{array} \right)$$

Eliminating the variable subset $\{x_1, x_2, x_3, x_4, x_5\}$ using the greedy algorithm follows the elimination order x_1, x_4, x_3, x_2, x_5 and yields 3,684 inequalities. By contrast, our heuristic produces the elimination order x_1, x_3, x_4, x_5, x_2 and yields only 1,680 inequalities, a reduction of 2,004 inequalities ($\approx 55.4\%$) compared with the greedy method.

A.3 Extensions of the Dynamic Programming Framework

Alg. 2 presents the dynamic programming framework for the CAD's Projection Phase. Alg. 3 presents the algorithm for extracting a variable elimination order from a nice tree decomposition.

Algorithm 2: A DP Framework for CAD's Projection Phase

Input : a NRA formula Φ of the form Eq. (1);
Output: a set of polynomials within $\mathbb{R}[x_{m+1}, \dots, x_n]$

```

1  $\triangleright$  Invoke external algorithms to compute nice tree-decompositions.
2 Construct the primal graph  $G_\Phi = (V_\Phi, E_\Phi)$  of  $\Phi$ ;
3 Compute a nice tree decomposition  $T = (\mathcal{B}, E_T)$  of  $G_\Phi$ ;
4  $\triangleright$  Initialize  $\mathcal{I}$  from top down.
5  $P \subset \mathbb{R}[\mathbf{x}] \leftarrow$  the set of polynomials in  $\Phi$ ;
6 Initialize an empty queue of bags  $q = \langle \rangle$ ;
7  $q.\text{enqueue}(r_T)$  ;
8 while  $q$  is not empty do
9    $b \leftarrow q.\text{dequeue}()$ ;
10   $q.\text{enqueue}(b.\text{children})$ ;
11  if  $b.\text{type} = \text{leaf}$  then
12     $\mathcal{I}(b) \leftarrow \emptyset$ ;
13  else
14     $\mathcal{I}(b) \leftarrow \{p \in P \mid \text{var}(p) \subseteq b\}$ ;
15     $P \leftarrow P \setminus \mathcal{I}(b)$ ;
16  end
17 end
18  $\triangleright$  Recursively compute  $\mathcal{V}$  from bottom up.
19  $q.\text{enqueue}(\mathcal{B}.\text{leaves})$  ;
20 while  $q$  is not empty do
21    $b \leftarrow q.\text{dequeue}()$ ;
22    $q.\text{enqueue}(b.\text{parent})$ ;
23   switch  $b.\text{type}$  do
24     case join do  $\mathcal{V}(b) \leftarrow \mathcal{I}(b) \cup \mathcal{V}(b.\text{leftchild}) \cup \mathcal{V}(b.\text{rightchild})$ ;
25     case introduce do  $\mathcal{V}(b) \leftarrow \mathcal{I}(b) \cup \mathcal{V}(b.\text{children})$ ;
26     case forget do  $\mathcal{V}(b) \leftarrow \mathcal{I}(b) \cup \text{Proj}(\bigwedge \mathcal{V}(b.\text{children}), b.\text{forget})$ ;
27     case leaf do  $\mathcal{V}(b) \leftarrow \mathcal{I}(b)$ ;
28   end
29 end
30 return  $\bigcup \mathcal{V}(r_T)$ ;
31  $\triangleright$  After this algorithm finishes, the CAD algorithm continues the
    projection phase for free variables, until only one variable is
    left. Then the lifting phase begins.

```

Algorithm 3: A General Framework for QE Procedures

Input : a $\mathcal{T}$ -formula Φ of the form Eq. (1) and a QE procedure QE;
Output: a quantifier-free $\mathcal{T}$ -formula equivalent to Φ

```

1 Construct the primal graph  $G_\Phi = (V_\Phi, E_\Phi)$  of  $\Phi$ ;
2 Compute a (nice) tree decomposition  $T = (\mathcal{B}, E_T)$  of  $G_\Phi$ ;
3  $\triangleright$  Extract a variable elimination order from  $T$ .
4 Initialize an empty queue of variables  $order = \langle \rangle$ ;
5 Initialize an empty queue of bags  $q = \langle \rangle$ ;
6  $q.enqueue(r_T)$ ;
7 while  $q$  is not empty do
8    $b \leftarrow dequeue(q)$ ;
9    $enqueue(q, b.children)$ ;
10  if  $b$  is root then
11     $order.enqueue(b)$ ;
12  else
13     $order.enqueue(b \setminus b.parent)$ 
14  end
15 end
16  $order \leftarrow order.reverse$ ; // reverse the ordering
17 return QE( $\Phi, order$ )

```

A.4 Sparsity in SMT-LIB benchmarks

We examined the QF_LRA and QF_NRA categories from the SMT-LIB benchmark library [5]. Using `pysmt`, we successfully parsed 1753 LRA instances and 9249 NRA instances. Among the parsed instances, only 337 LRA instances and 807 NRA instances have non-complete primal graphs. We first analyzed the treewidth of these instances, the results are presented in Table 3 and Table 4. However, since many instances contain hundreds or more variables, it is impractical to perform a full tree decomposition for every instance (e.g., within a 10-minute timeout). Therefore, we also examined the edge density sparsity, defined by $|E|/\binom{|V|}{2}$, where E and V denote the number of edges and vertices of a primal graph, respectively. The results are presented in Table 5 and Table 6. From these results, we found that SMT-LIB benchmarks are generally too large and too dense for demonstrating our algorithms.

Table 3: Treewidth distribution among 337 sparse instances in the QF_LRA dataset

Treewidth range	Number	% in sparse instances	Min #var	Max #var	Median #var
(0, 10]	1	0.27	70	70	70.0
(10, 20]	47	12.47	56	8954	144.0
(20, 40]	98	25.99	102	9937	2019.5
(40, 60]	31	8.22	259	7242	1307.0
(60, 80]	21	5.57	223	4467	1173.0
(80, 100]	21	5.57	315	7793	1433.0
(100, 200]	58	15.38	425	4855	2065.0
(200, 250]	5	1.33	1291	2452	1881.0
Timeout	95	0.25	NA	NA	NA

Table 4: Treewidth distribution among 807 sparse instances in the **QF_NRA** dataset

Treewidth range	Number	% in sparse instances	Min #var	Max #var	Median #var
(0, 5]	11	1.36	6	248	6.0
(5, 10]	7	0.87	76	424	330.0
(10, 20]	20	2.48	33	8220	255.5
(20, 40]	294	36.43	37	9189	467.0
(40, 60]	153	18.96	68	9352	680.0
(60, 80]	109	13.51	102	6068	887.0
(80, 100]	53	6.57	126	3441	1167.0
(100, 200]	89	11.03	124	5149	1183.0
(200, 300]	7	0.87	279	1541	380.0
(300, 550]	2	0.25	605	756	680.5
Timeout	62	0.08	NA	NA	NA

Table 5: Edge density among 337 sparse instances in the **QF_LRA** dataset

Density range	Number	% in sparse instances	Min #var	Max #var	Median #var
(0.0, 0.05]	225	59.68	307	9937	2864.0
(0.05, 0.1]	50	13.26	147	9385	365.5
(0.1, 0.15]	37	9.81	83	8622	204.0
(0.15, 0.2]	35	9.28	61	6416	132.0
(0.2, 0.25]	6	1.59	56	444	174.0
(0.3, 0.35]	8	2.12	1617	1617	1617.0
(0.85, 0.9]	1	0.27	1785	1785	1785.0
(0.9, 0.95]	4	1.06	1274	4985	2435.0
(0.95, 1.0]	11	2.92	1798	4285	1798.0

Table 6: Edge density among 807 sparse instances in the **QF_NRA** dataset

Density range	Number	% in sparse instances	Min #var	Max #var	Median #var
(0.0, 0.05]	418	51.80	122	9929	2025.0
(0.05, 0.1]	189	23.42	123	9525	441.0
(0.1, 0.15]	50	6.20	57	3309	215.0
(0.15, 0.2]	23	2.85	28	4052	249.0
(0.2, 0.25]	2	0.25	154	154	154.0
(0.25, 0.3]	4	0.50	110	756	190.0
(0.3, 0.35]	15	1.86	68	282	168.0
(0.35, 0.4]	9	1.12	70	282	130.0
(0.4, 0.45]	16	1.98	59	1678	175.0
(0.45, 0.5]	14	1.73	70	216	95.5
(0.5, 0.55]	17	2.11	51	303	86.0
(0.55, 0.6]	16	1.98	33	876	115.5
(0.6, 0.65]	9	1.12	68	1136	208.0
(0.65, 0.7]	4	0.50	6	178	104.5
(0.7, 0.75]	5	0.62	172	702	605.0
(0.75, 0.8]	9	1.12	6	1925	6.0
(0.8, 0.85]	2	0.25	124	198	161.0
(0.85, 0.9]	3	0.37	132	3525	2602.0
(0.9, 0.95]	2	0.25	3802	5125	4463.5

A.5 Additional Experiments

We conducted experiments on benchmarks of the lattice reachability problems from [53, Example. 9], and the results are presented in Table 7. Note that, since the primal graphs exhibit a special "straight-line" structure, the variable elimination orderings generated by our methods coincide with those of [51], and hence the experimental results also coincide.

Table 7: CAD performance on lattice reachability problems over **Mathematica**

ID	Instance	SVO	Brown	PEO/Ours
13	#var=11 #ineq=8 Cells	5,299	6,067	5,299
	#max_deg=2 #tw=3 Time	2.5	1.1	2.5
14	#var=12 #ineq=9 Cells	14,509	24,141	14,509
	#max_deg=2 #tw=3 Time	6.8	5.4	7.4
15	#var=13 #ineq=10 Cells	39,625	65,961	39,625
	#max_deg=2 #tw=3 Time	20.8	15.8	23.4
16	#var=14 #ineq=11 Cells	127,195	185,515	109,787
	#max_deg=2 #tw=3 Time	25.6	43.5	53.6
17	#var=15 #ineq=12 Cells	511,703	303,967	303,967
	#max_deg=2 #tw=3 Time	175.1	194.3	142.0
18	#var=16 #ineq=13 Cells	3,113,905	847,351	847,351
	#max_deg=2 #tw=3 Time	1565.2	430.7	432.9
19	#var=17 #ineq=14 Cells	NA	2,362,235	2,362,235
	#max_deg=2 #tw=3 Time	> 3h	1436.6	1399.5

A.6 The NRA Formulas in Table 2

The randomly generated LRA formulas can be found via the link to our code. The randomly generated sparse NRA formulas in Table 2 are shown here.

$$\Phi_7 \triangleq \exists x_1, x_2, x_3, x_4, x_5, x_6. \left(\begin{array}{l} 8x_1x_2 + 6x_1 + 5x_3 + 5 \geq 0 \\ \wedge -4x_1x_3 - 10x_4^2 + 4x_4 + 9 \geq 0 \\ \wedge 6x_3x_5 - 3x_4x_5 + 4x_4^2 + 8x_5^2 - 6 \geq 0 \\ \wedge -5x_4x_5 + 7x_4x_6 + x_5^2 - 5 \geq 0 \\ \wedge -2x_2^2 + 6 \geq 0 \\ \wedge -10x_1x_3 - 4x_3^2 + 5x_2 - 7x_3 + 2 \geq 0 \end{array} \right)$$

$$\Phi_8 \triangleq \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7. \left(\begin{array}{l} -8x_1x_2^2 + 5x_1x_3^2 + x_4^2 + 7 \geq 0 \\ \wedge 7x_1x_2x_4 - 9x_2^2 - 2x_5^2 + 5 \geq 0 \\ \wedge -2x_1^2x_6 - 6x_4x_5 - 8 \geq 0 \\ \wedge -9x_2x_7^2 + 6x_1x_4 - 9 \geq 0 \end{array} \right)$$

$$\begin{aligned}\Phi_9 &\doteq \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7. \\ &\left(\begin{array}{l} 7x_1x_3 + 2x_2x_3 + 7x_2^2 - 8x_3^2 + 4x_2 + 10x_3 \geq 0 \\ \wedge -5x_2x_3 - x_3x_5 + 7x_5^2 - 3x_2 \geq 0 \\ \wedge 7x_2^2 + 3x_4^2 + 4x_5^2 - 10x_4 + 6x_5 \geq 0 \\ \wedge -x_1x_6 + 2x_1^2 + 4x_6 \geq 0 \\ \wedge 9x_4x_7 - 10x_4 \geq 0 \end{array} \right)\end{aligned}$$

$$\begin{aligned}\Phi_{10} &\doteq \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8. \\ &\left(\begin{array}{l} 10x_1^2x_2x_3 + 8x_1^4 - 7x_1x_2^2 + 8x_1^2x_2 + 10x_1^2 + 10x_2^2 \geq 0 \\ \wedge 5x_3x_4^3 + 10x_4^4 + 4x_3x_5^2 + 2x_5^3 \geq 0 \\ \wedge 5x_3x_4^2x_7 - 4x_3^2x_4 - 9x_4^3 + 4x_3x_7 + 4x_3 + 8x_7 \geq 0 \\ \wedge 2x_4x_5^2x_6 + 2x_4^3x_6 + 3x_4^2x_5 + 4x_4^3 + 7x_4x_6 - 10x_5 \geq 0 \\ \wedge 10x_4x_8^3 - 3x_4x_8^2 - 9x_4^3 + 3x_8^2 \geq 0 \end{array} \right)\end{aligned}$$

$$\begin{aligned}\Phi_{11} &\doteq \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8. \\ &\left(\begin{array}{l} 10x_1x_2 + 2x_2x_3 + 4x_3x_4 + 6 \geq 0 \\ \wedge -x_1x_3 - 9x_2x_3 + 9x_2x_5 + 9 \geq 0 \\ \wedge 3x_1x_6 - 7x_2x_4 - 2x - 6^2 - 3 \geq 0 \\ \wedge -5x_2x_6 + 7x_2x_7 + 8x - 4x_6 + 4x_4 - 10x_7 - 9 \geq 0 \\ \wedge -6x_1x_4 - 3x_3x_4 + 3x_4x_8 + 2x_1 - 10 \geq 0 \\ \wedge 7x_2^2 - 2 \geq 0 \end{array} \right)\end{aligned}$$

$$\begin{aligned}\Phi_{12} &\doteq \exists x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9. \\ &\left(\begin{array}{l} -x_1x_4 + 5x_1^2 + x_2x_3 + 9 \geq 0 \\ \wedge -8x_1^2 - 5x_2^2 - 8x_3x_5 - 10x_3 + 7 \geq 0 \\ \wedge -9x_1x - 2 - 6x_1x_5 - 3x_6^2 - 9x_1 - 9 \geq 0 \\ \wedge 7x_1x_7 + 10x_2x_5 + 4x_2^2 - 3 \geq 0 \\ \wedge -8x_1x_2 + 7x_5x_8 + 8 \geq 0 \\ \wedge 9x_3x_5 + 8x_3x_9 + 2x_1 - 5 \geq 0 \\ \wedge 2x_1^2 - 2x_2^2 + 5x_5x_7 + 1 \geq 0 \\ \wedge x_2x_4 + 1 \geq 0 \\ \wedge -3x_3^2 + 5 \geq 0 \\ \wedge -3x_1x_2 - 6x_1^2 - 4 \geq 0 \end{array} \right)\end{aligned}$$